

Placement Portal Application Project Report

1. Student Details

Name: Md Ayazuddin

Roll Number: 24f1002435

Email: 24f1002435@ds.study.iitm.ac.in

About Me: I am a student in the IIT Madras BS Degree program, currently in the Diploma stage. I enjoy building simple web applications and improving my coding skills. Through this project, I learned more about Python, Flask, HTML, and database design.

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

Institutes need efficient systems to manage campus recruitment activities involving companies, students, and placement drives. Currently, many institutes rely on spreadsheets, emails, or manual processes, which makes it difficult to manage company approvals, track student applications, avoid duplicate registrations, and maintain placement records.

You are required to build a Placement Portal Application web application that allows Admin (Institute), Company, and students to interact with the system based on their roles.

Approach:

The app was built using Flask as the backend framework with a modular structure separating models, routes, and templates. It supports three roles Admin, Company, and Student each with their own dashboard and access controls. The database was designed using SQLAlchemy with SQLite, and all data is created programmatically via a seed script.

3. AI/LLM Declaration

I used Claude (Anthropic) LLM to assist variable naming and documentation formatting. The extent of AI/LLM usage is approximately 5%. All core implementation logic, database design, and final integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
werkzeug.security	For Password Hash
Flask-Login	User authentication and session management
Werkzeug.Utils	For File/Pdf Upload
SQLite	Lightweight local database for storing user data

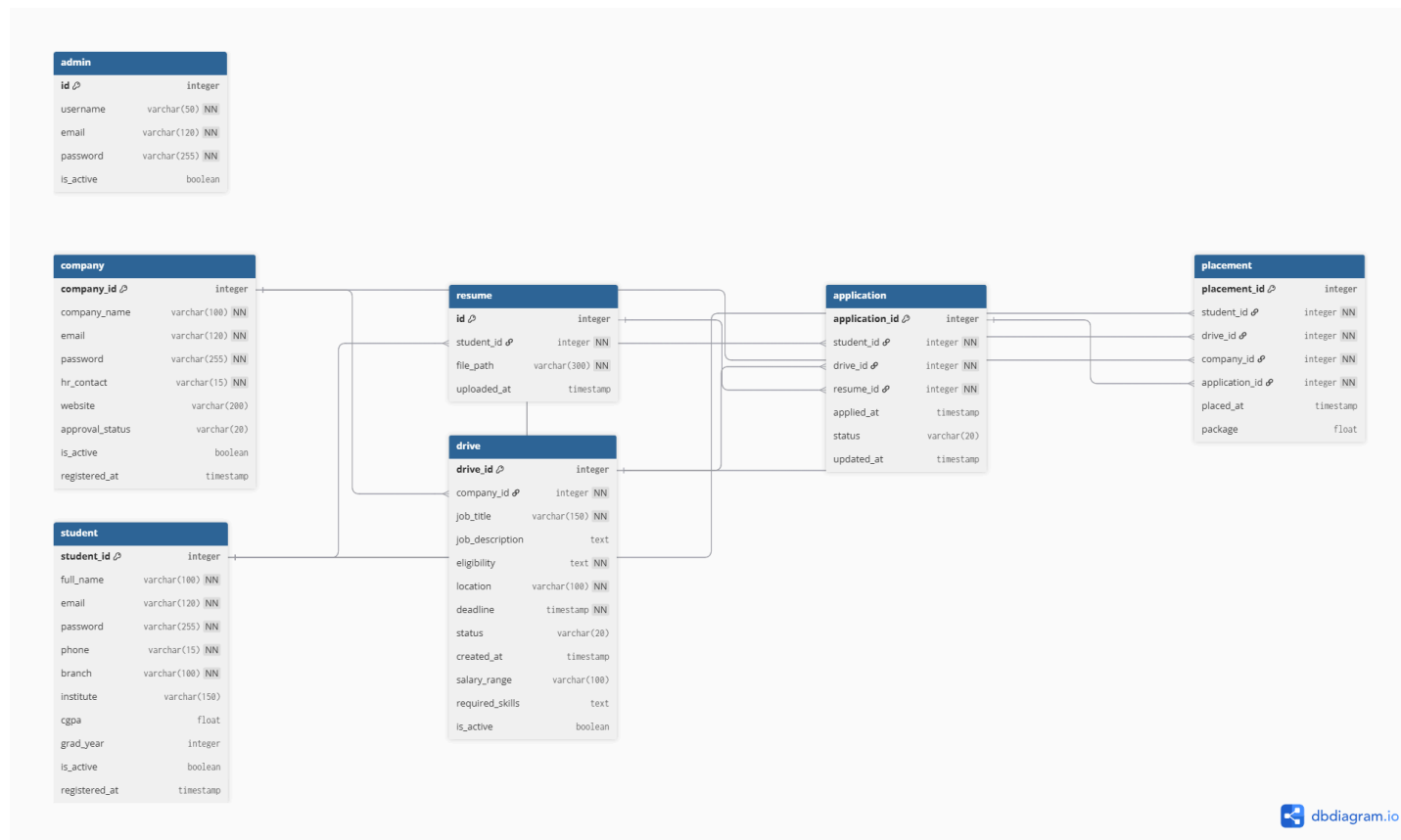
5. Database Schema / ER Diagram

Tables:

1. **Admin** — stores the predefined superuser credentials (id, username, email, password, is_active)
2. **Company** — stores registered company profiles (company_id, company_name, email, password, hr_contact, website, approval_status, is_active, registered_at)
3. **Student** — stores registered student profiles (student_id, full_name, email, password, phone, branch, institute, cgpa, grad_year, is_active, registered_at)
4. **Resume** — stores uploaded resume file path linked to a student (id, student_id, file_path, uploaded_at)
5. **Placement_Drive** — stores placement drives created by companies (drive_id, company_id, job_title, job_description, eligibility, location, deadline, status, salary_range, required_skills, is_active, created_at)
6. **Application** — records a student applying to a placement drive (application_id, student_id, drive_id, resume_id, status, applied_at, updated_at)

Relationships:

- One-to-Many → Company → Placement_Drive
- One-to-Many → Placement_Drive → Application
- One-to-Many → Student → Application
- One-to-One → Student → Resume
- Many-to-One → Application → Resume
- One-to-Many → Company → Placement
- One-to-Many → Student → Placement



(taken from dbdiagram.io)

6. API Resource Endpoints

Here are your API Resource Endpoints in the same format:

API Resource Endpoints

Endpoint	Method	Description
<code>/login</code>	POST	Authenticate user (Admin, Company, Student) and create session
<code>/logout</code>	GET	End current user session
<code>/register/student</code>	POST	Register a new student account
<code>/register/company</code>	POST	Register a new company account
<code>/dashboard/admin</code>	GET	Fetch admin dashboard data (companies, students, drives, applications)
<code>/dashboard/student</code>	GET	Fetch student dashboard (drives, applications, notifications)
<code>/dashboard/company</code>	GET	Fetch company dashboard (drives, applicants)
<code>/dashboard/drive/<drive_id></code>	GET	Fetch details of a specific placement drive
<code>/dashboard/application/<application_id></code>	GET	Fetch details of a specific application
<code>/dashboard/student/history</code>	GET	Fetch complete application history for logged-in student
<code>/student/apply/<drive_id></code>	GET	Submit application for a placement drive
<code>/application/status/<application_id>/<status></code>	GET	Update application status (Shortlisted / Selected / Rejected etc.)
<code>/admin/company/approve/<company_id></code>	GET	Approve a company registration
<code>/admin/company/reject/<company_id></code>	GET	Reject a company registration

<code>/admin/company/blacklist/<company_id></code>	GET	Blacklist an approved company
<code>/admin/company/delete/<company_id></code>	GET	Permanently delete a company
<code>/admin/company/edit/<company_id></code>	GET / POST	View and update company details
<code>/admin/student/edit/<student_id></code>	GET / POST	View and update student details
<code>/admin/student/blacklist/<student_id></code>	GET	Blacklist a student account
<code>/admin/student/delete/<student_id></code>	GET	Permanently delete a student
<code>/admin/drive/approve/<drive_id></code>	GET	Approve a placement drive
<code>/admin/drive/reject/<drive_id></code>	GET	Reject a placement drive
<code>/company/new_drive</code>	GET / POST	Create a new placement drive
<code>/company/edit_drive/<drive_id></code>	GET / POST	Edit an existing placement drive
<code>/company/view_applicants/<drive_id></code>	GET	View all applicants for a drive
<code>/dashboard/student/edit</code>	GET / POST	Student edits their own profile and uploads resume
<code>/drive/complete/<drive_id></code>	GET	Mark a placement drive as complete
<code>/admin/search</code>	GET	Search students and companies
<code>/student/search</code>	GET	Search drives and companies

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** - main Flask application entry point and app factory
- **backend/models.py** - database models using SQLAlchemy
- **backend/routes.py** - all Flask route handlers
- **backend/create_initialdata.py** — database seeding script with sample data
- **templates/** — Jinja2 HTML templates organized by role (admin/, company/, student/)
- **static/resumes/** — uploaded student PDF resumes
-

Implemented Features:

- User registration and login for Student and Company with password hashing
 - Duplicate application prevention at route, UI, and database level
 - Application status tracking (Applied / Shortlisted / Interview / Waiting / Selected / Rejected)
 - Student notification system for application status changes
 - Complete application history for each student
 - Search functionality for admin (students and companies) and students (drives and companies)
 - Blacklist and delete system for students and companies
 - Confirmation modals before destructive actions (delete, blacklist)
 - Responsive UI built with Bootstrap 5
-

8. Video Presentation

Drive Link

https://drive.google.com/file/d/1jbSAP_g6KuyuSyiVYTtcPguFy2f4UuE_/view?usp=sharing